

成绩

南京工程学院

期末大作业课题说明书（论文）

题 目	智慧校园外卖点餐系统
课 程 名 称	WEB 应用开发技术课程设计
院（系、部、中心）	计算机工程学院
专 业	软件工程
班 级	软工中外 232
学 生 姓 名	陈天旸
学 号	21230215

说明：本报告为完整电子版，正文、图表、系统截图和关键代码分析均已整理。

报告结构说明

本报告依据《WEB 开发技术》期末大作业任务书和课程设计报告模板编写，围绕“Java Web 外卖点餐系统”选题展开。报告内容覆盖系统设计目的、需求分析、开发技术、功能模块、数据库设计、Web 软件系统设计、调试测试、课程收获与参考资料。

由于本项目实际采用 Spring Boot + MyBatis + React 前后端分离架构，报告在传统 JSP/Servlet/JDBC/Ajax 要求基础上，说明了使用现代 Java Web 框架完成同等业务目标的原因：后端仍由 Java Web 服务提供接口与数据库访问，前端通过 Axios 发起异步请求并渲染页面，能够更清晰地体现数据库动态数据驱动页面的过程。

一、系统设计目的

本系统的设计目的，是围绕校园外卖点餐场景实现一个具有完整业务闭环的 Web 应用。系统面向游客、客户和管理员三类角色：游客可以浏览和搜索菜品，客户登录后可以加入购物车、提交订单并查询订单进度，管理员可以维护菜品分类、菜品信息、订单状态和库存数据。

在实现过程中，我重点考虑了课程要求中的页面交互、数据库动态数据、Ajax 获取数据和 Java Web 后端业务处理。后端使用 Spring Boot 组织 Controller、Service、Mapper 分层逻辑，MyBatis 负责数据库访问；前端使用 React 构建页面，通过 Axios 与后端接口通信。这样做的原因是前后端职责清晰，接口数据结构明确，既便于调试，也便于在报告中说明每个功能模块的实现过程。

作为个人独立完成的课程设计，本项目不仅完成基本的浏览、下单和管理功能，还加入了订单状态时间轴、热销推荐、库存预警、数据看板和“已有订单菜品只能下架不能删除”等业务规则，使系统更接近真实外卖点餐平台。

二、系统需求分析

根据任务书要求，本项目选择“Java Web 外卖点餐系统”。系统用于餐饮企业或校园食堂实现外卖点餐业务，用户分为管理员、客户、游客。需求分析时，我将功能拆分为前台点餐业务和后台管理业务两部分，并保证所有页面核心数据来源于数据库。

角色	主要需求	权限边界
游客	浏览首页、查看菜品分类、模糊	不能下单，加入购物车时跳转登

	查询菜名、查看菜品详情	录
客户	注册登录、维护个人信息、加入购物车、提交订单、查询订单进度、取消待接单订单	只能查看和操作自己的订单
管理员	分类管理、菜品管理、图片上传、订单管理、库存预警、销售统计	可查看全部订单并推动订单状态流转

表 1 用户角色与需求分析

智慧校园外卖点餐系统用例图

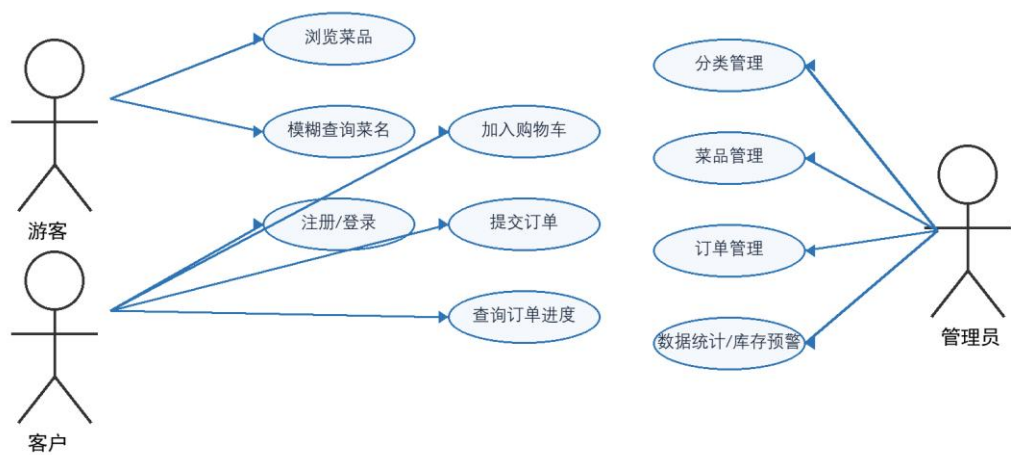


图 1 系统用例图

用例图体现了三类用户和系统之间的主要交互。游客侧重点是浏览和搜索；客户侧重点是点餐和订单跟踪；管理员侧重点是后台维护与经营管理。这样划分可以让权限控制更清楚，也避免客户直接访问后台数据。

客户下单业务流程图

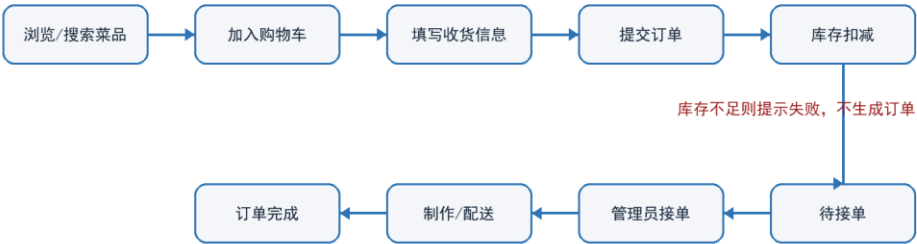


图 2 客户下单业务流程图

下单流程中最关键的业务规则是库存校验和状态流转。客户提交订单时系统会根据购物车明细计算总价，并扣减对应菜品库存；订单生成后状态为待接单，管理员按接单、制作、配送、完成的顺序推动订单状态。若客户在待接单阶段取消订单，系统会恢复库存和销量。

三、开发技术简介

本项目采用 Spring Boot、MyBatis、MySQL、React、Vite、Axios、TanStack Query、JWT 等技术完成。选择前后端分离方案，是因为外卖点餐系统有较多列表、筛选、状态刷新和表单交互，前端独立实现页面交互会更灵活；后端通过 RESTful API 输出 JSON 数据，能够清晰体现 Ajax 异步获取数据并渲染页面的过程。

技术	用途	采用原因
Spring Boot	后端 Web 服务、接口路由、参数校验	简化 Java Web 配置，便于快速实现 Controller/Service 分层
MyBatis	数据库访问和 SQL 映射	SQL 可控，适合展示课程设计中的数据表关系和查询逻辑
MySQL	保存用户、分类、菜品、购物车和订单数据	关系模型清晰，适合 ER 图和二维关系表设计
React + Vite	前端页面和交互	组件化开发，页面跳转、状态刷新和表单处理更清楚

Axios / React Query	Ajax 请求、数据缓存和刷新	页面数据全部从后端接口获取，符合动态渲染要求
JWT	登录状态和接口权限	前后端分离场景下便于携带身份信息并限制后台接口

表 2 开发技术及选择原因

虽然任务书中列出了 JSP、Servlet、JDBC、JQuery、JSTL 等传统 Java Web 技术点，本项目采用的是更高层的 Spring Boot 和 React 实现。Spring Boot 中的 Controller 可以看作对 Servlet 请求处理的工程化封装，MyBatis 完成 JDBC 数据库访问的封装，React 与 Axios 完成 JQuery/Ajax 类似的异步交互。这样既保留了 Java Web 的核心请求-处理-数据库-响应链路，也提升了系统可维护性。

四、系统功能模块图

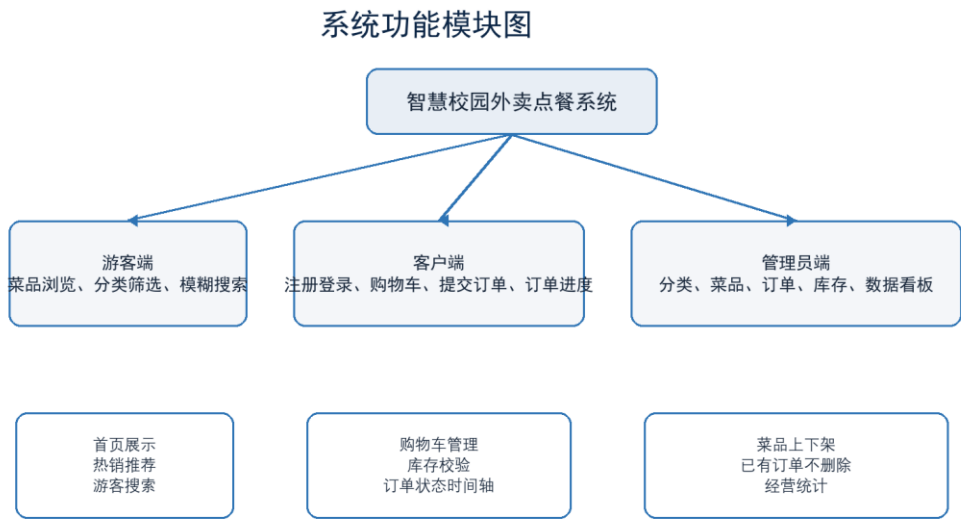


图 3 系统功能模块图

系统模块分为游客端、客户点餐端和管理员后台。游客端提供浏览和搜索入口；客户点餐端完成购物车、订单提交和进度查询；管理员后台完成分类、菜品、订单、库存和统计管理。我的实现重点放在后端业务规则和接口设计，同时通过前端页面将功能完整展示出来。

- 游客端：降低使用门槛，未登录也可以浏览菜品和分类，体现外卖平台的公开展示属性。
- 客户端：围绕点餐闭环设计，从加入购物车到订单状态时间轴，让用户知道订单当前进度。
- 管理员端：围绕运营管理设计，重点是分类维护、菜品上下架、订单处理和库存预警。

五、系统的详细设计

1、数据库设计

数据库设计围绕用户、菜品、购物车、订单四条主线展开。用户表保存账号和角色，分类表与菜品表构成菜品展示基础，购物车表和购物车明细表保存客户临时点餐数据，订单表和订单明细表保存最终交易数据，订单状态日志表用于记录订单每次状态变化。

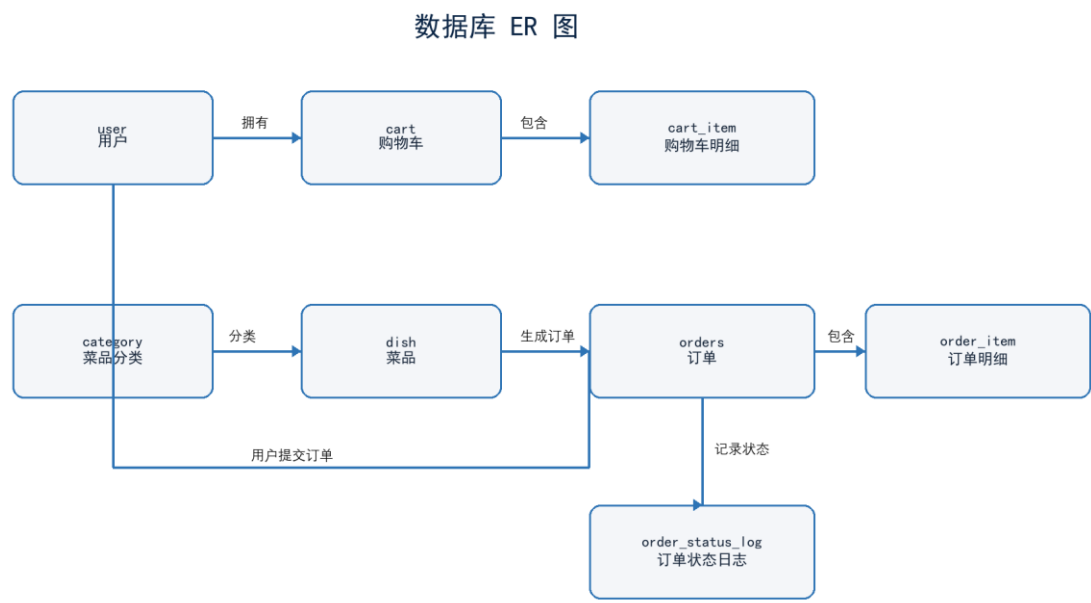


图 4 数据库 ER 图

表名	主要字段	设计说明
user	id、username、password、phone、role、address	保存客户和管理员账号，role 字段区分权限
category	id、name、sort_order	保存菜品分类，后台可新增、修改、删除
dish	id、category_id、name、price、stock、image_url、status	保存菜品信息，status 控制上架/下架
cart / cart_item	user_id、dish_id、quantity	保存客户购物车和菜品数量
orders	order_no、user_id、total_amount、receiver_*、	保存订单主信息、收货信息和备注

	status、remark	
order_item	order_id、dish_id、 dish_name、dish_price、 quantity	保存订单菜品快照，避免菜品后续修改影响历史订单
order_status_log	order_id、status、 operator_id、remark	记录订单状态变化，用于订单进度时间轴

表 3 数据库逻辑关系表

其中 order_item 保存的是菜品名称和价格快照，这样即使管理员之后修改菜品名称或价格，历史订单仍然保持下单时的真实信息。已有订单引用过的菜品不允许直接删除，而是改为下架，这也是为了保护订单明细的完整性。

2、WEB 软件系统设计

系统采用前后端分离结构。前端路由负责页面跳转，后端接口负责业务处理，数据库保存状态。后端接口统一返回 JSON，前端通过 Axios 请求后将数据渲染为页面。下面结合界面截图和关键代码说明主要模块。

模块	主要接口/页面	实现说明
认证模块	/api/auth/register、 /api/auth/login	注册区分客户和管理员，管理员需输入预设码 yangyang
菜品模块	/api/dishes、 /api/admin/dishes	支持分类筛选、模糊查询、推荐、上下架、图片上传
购物车模块	/api/cart	客户登录后维护购物车，结算时从购物车生成订单
订单模块	/api/orders、 /api/admin/orders	客户查自己的订单，管理员查全部订单并修改状态
统计模块	/api/admin/statistics/*	为后台数据看板、热销排行和库存预警提供数据

表 4 主要接口与模块说明

(1) 首页、菜品浏览与模糊查询



图 5 系统首页：展示菜品分类、今日推荐和热销榜

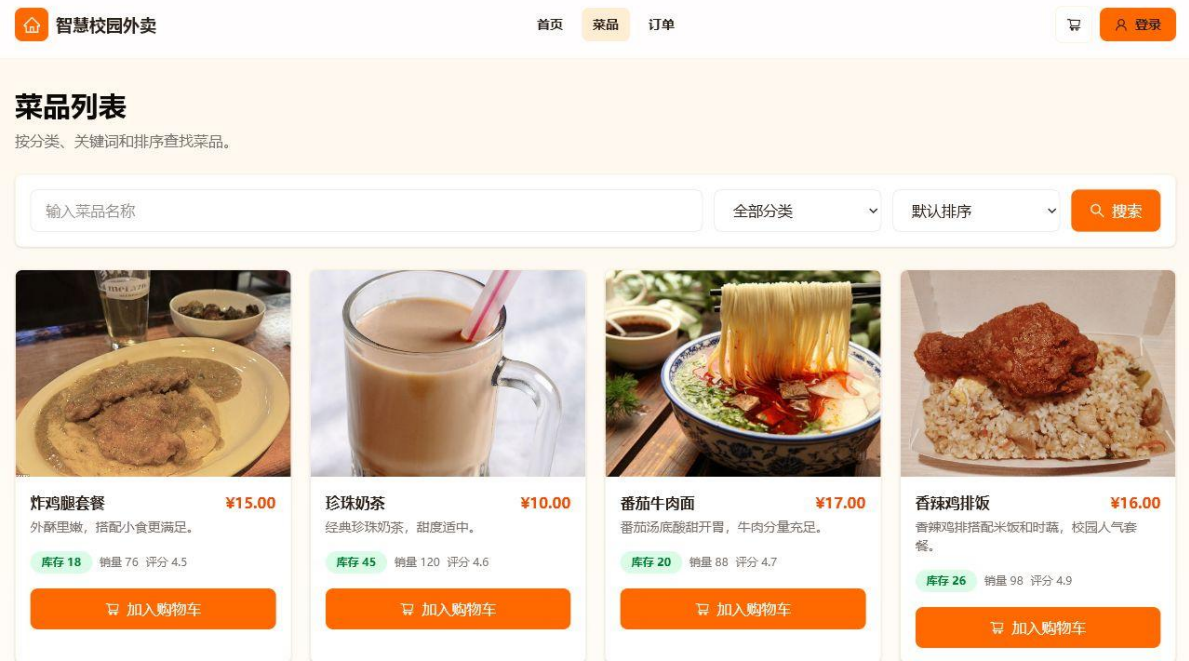


图 6 菜品列表：支持关键词、分类和排序筛选

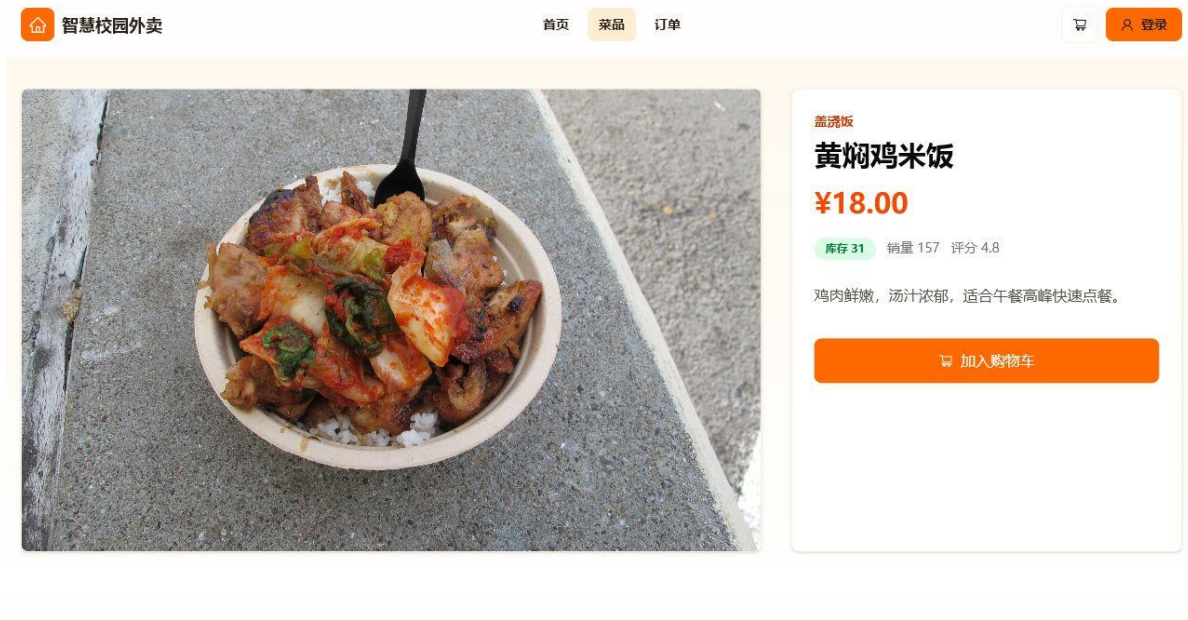


图 7 菜品详情：显示图片、价格、库存、分类和加入购物车入口

首页和菜品页的数据均来自后端数据库。前端通过查询参数传递 keyword、categoryId、sort 等条件，后端 MyBatis 根据条件动态拼接查询语句。这样做的原因是筛选逻辑集中在后端，前端只负责展示和交互，后续扩展价格排序、销量排序或分页时更方便。

```
@GetMapping
public ApiResponse<PageResult<DishVO>> list(DishQuery query) {
    return ApiResponse.success(dishService.list(query));
}

@Select("""
SELECT d.*, c.name AS category_name
FROM dish d LEFT JOIN category c ON c.id = d.category_id
<where>
    <if test="keyword != null and keyword != ''">
        AND d.name LIKE CONCAT('%', #{keyword}, '%')
    </if>
    <if test="categoryId != null">AND d.category_id = #{categoryId}</if>
</where>
""")
List<Dish> findPage(DishQuery query);
```

(2) 注册登录与权限控制

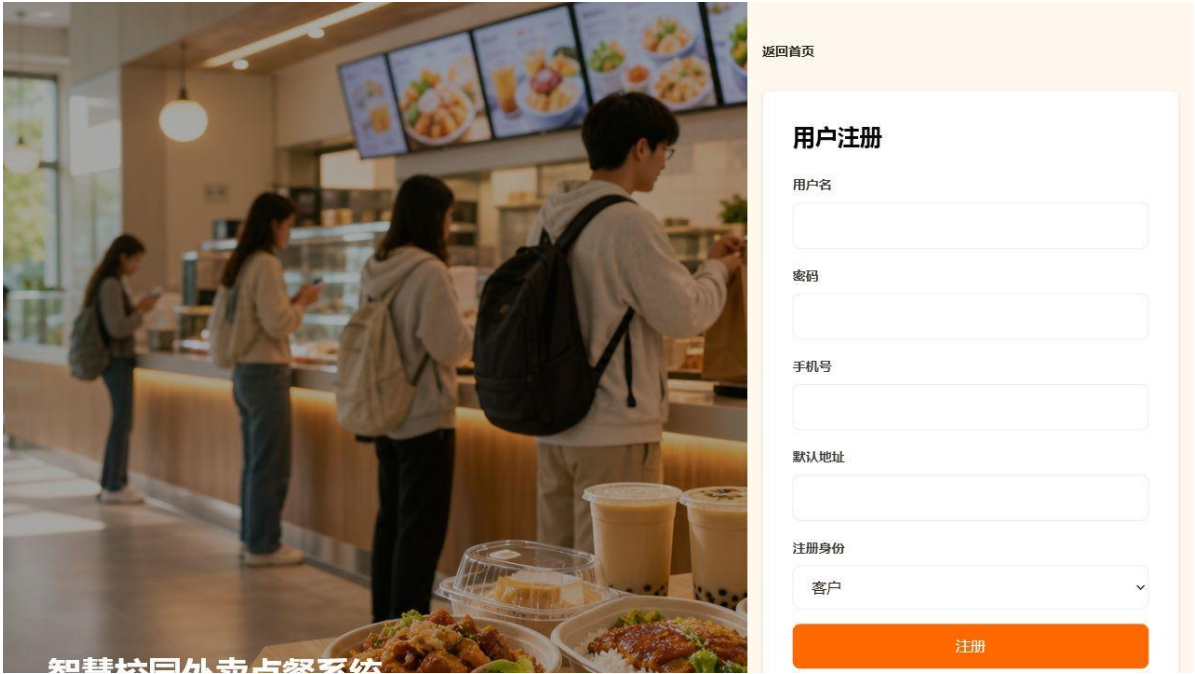


图 8 客户注册界面

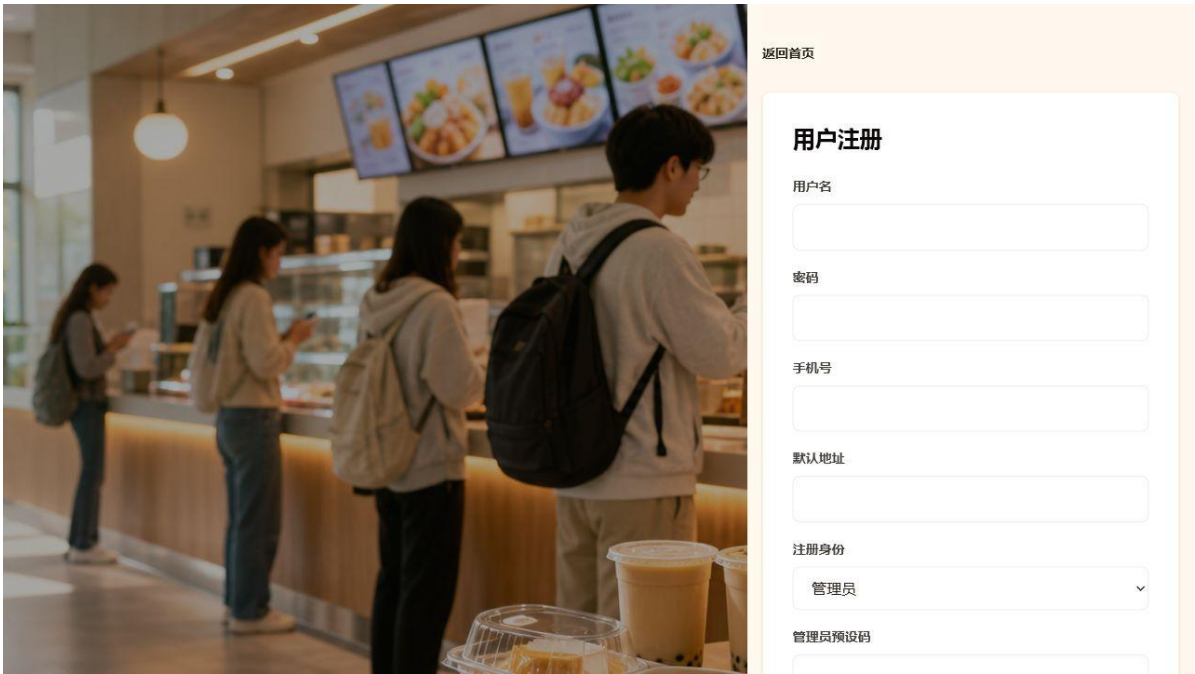


图 9 管理员注册界面：选择管理员后显示预设码输入框

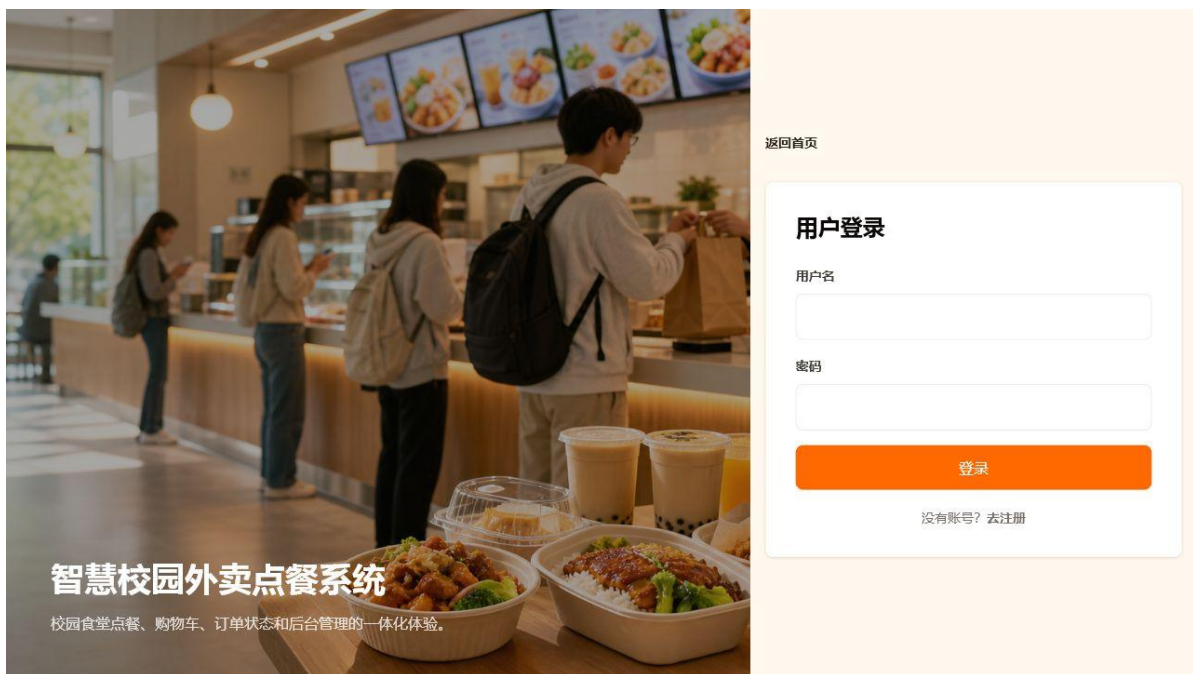


图 10 登录界面

注册功能按照任务书要求区分管理员和客户。普通客户可以直接注册；管理员注册必须输入预设码 yangyang。这个校验放在后端而不是只放在前端，是为了防止用户绕过页面直接调用接口创建管理员账号。

```
private String resolveRegisterRole(RegisterRequest request) {
    String role = StringUtils.hasText(request.getRole())
        ? request.getRole().trim().toUpperCase()
        : ROLE_CUSTOMER;
    if (ROLE_CUSTOMER.equals(role)) return ROLE_CUSTOMER;
    if (ROLE_ADMIN.equals(role)) {
        if (!ADMIN_REGISTER_CODE.equals(request.getAdminCode())) {
            throw new BusinessException(400, "管理员注册码错误");
        }
        return ROLE_ADMIN;
    }
    throw new BusinessException(400, "注册角色无效");
}
```

登录成功后后端生成 JWT，前端保存 token 并在后续请求头中携带。后台接口会通过 `CurrentUserUtil.requireAdmin` 校验角色，避免普通客户访问管理端接口。

(3) 购物车、结算与订单进度



图 11 购物车界面：客户确认菜品数量并进入结算



图 12 结算界面：填写收货人、电话、地址和备注



图 13 我的订单：客户查看自己的订单列表

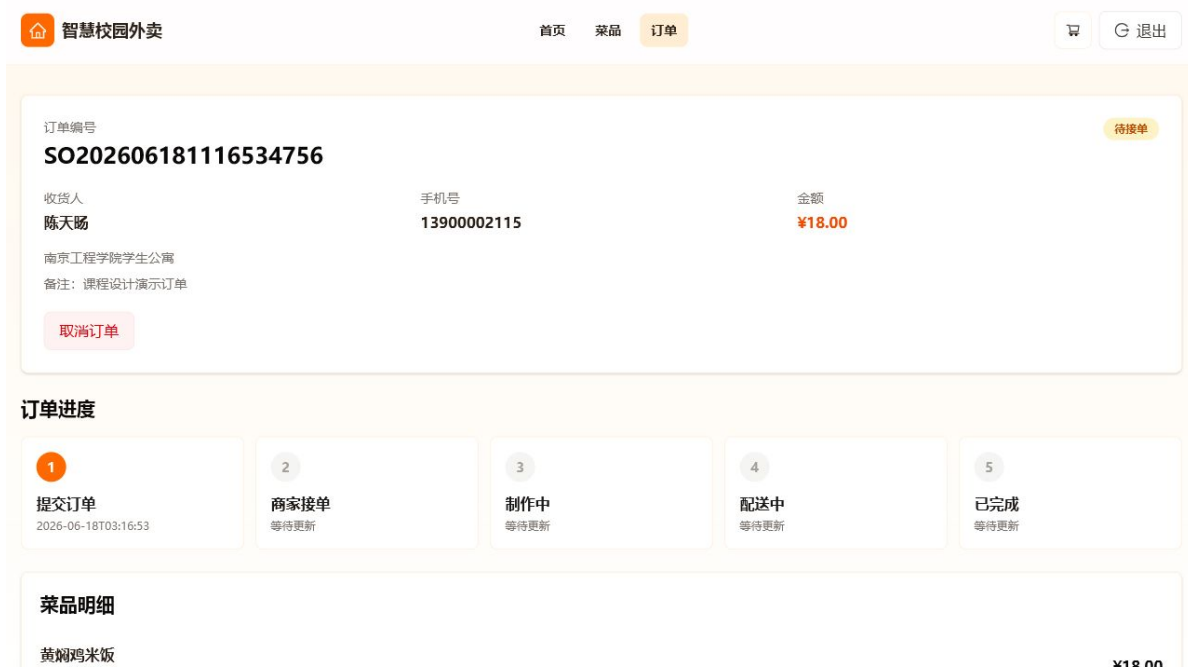


图 14 订单详情：展示订单进度时间轴和菜品明细

订单模块是系统的核心业务。客户提交订单时，系统从购物车读取菜品明细，校验菜品是否上架和库存是否足够，计算总金额，写入订单主表和订单明细表，并记录第一条订单状态日志。

```
@Transactional
public CreateOrderVO create(Long userId, CreateOrderRequest request) {
    List<CartItem> cartItems = cartMapper.findItemsByUserId(userId);
    validateCartItems(cartItems);
```

```

Order order = new Order();
order.setOrderNo(generateOrderNo());
order.setUserId(userId);
order.setTotalAmount(totalAmount(cartItems));
orderMapper.insertOrder(order);
addStatusLog(order.getId(), "PENDING", userId, "CUSTOMER", "提交订单");
for (CartItem item : cartItems) {
    dishMapper.decreaseStock(item.getDishId(), item.getQuantity());
    dishMapper.increaseSales(item.getDishId(), item.getQuantity());
    orderMapper.insertOrderItem(toOrderItem(order.getId(), item));
}
cartMapper.clearByUserId(userId);
return toCreateOrderVO(order);
}

```

订单进度使用 order_status_log 表记录。这样做的好处是订单当前状态和历史状态分离，前端可以根据日志生成时间轴，而不是只显示一个状态文本。

(4) 管理员后台管理



图 15 管理端数据看板：展示订单、销售额、客户数、热销菜品等统计

管理后台

数据看板

分类管理

菜品管理

订单管理

库存预警

返回前台

退出

分类管理

分类下已有菜品时后端会阻止删除

分类名称

0

新增分类

ID	名称	排序	菜品数	操作
1	盖浇饭	1	2	
2	面食	2	1	
3	饮品	3	1	
4	小吃	4	1	

图 16 分类管理：新增、修改、删除菜品分类

管理后台

数据看板

分类管理

菜品管理

订单管理

库存预警

返回前台

退出

菜品管理

已有订单菜品删除失败时改为下架

菜品名称

选择分类

价格

库存

图片地址

上架

上传图片

新增菜品

菜品介绍

菜品	分类	价格	库存	状态	销量	操作
炸鸡腿套餐	小吃	¥15.00	库存 18	上架	76	下架
珍珠奶茶	饮品	¥10.00	库存 45	上架	120	下架
番茄牛肉面	面食	¥17.00	库存 20	上架	88	下架
香辣鸡排饭	盖浇饭	¥16.00	库存 26	上架	98	下架
黄焖鸡米饭	盖浇饭	¥18.00	库存 31	上架	157	下架

图 17 菜品管理：维护菜品分类、价格、库存、图片和上下架状态

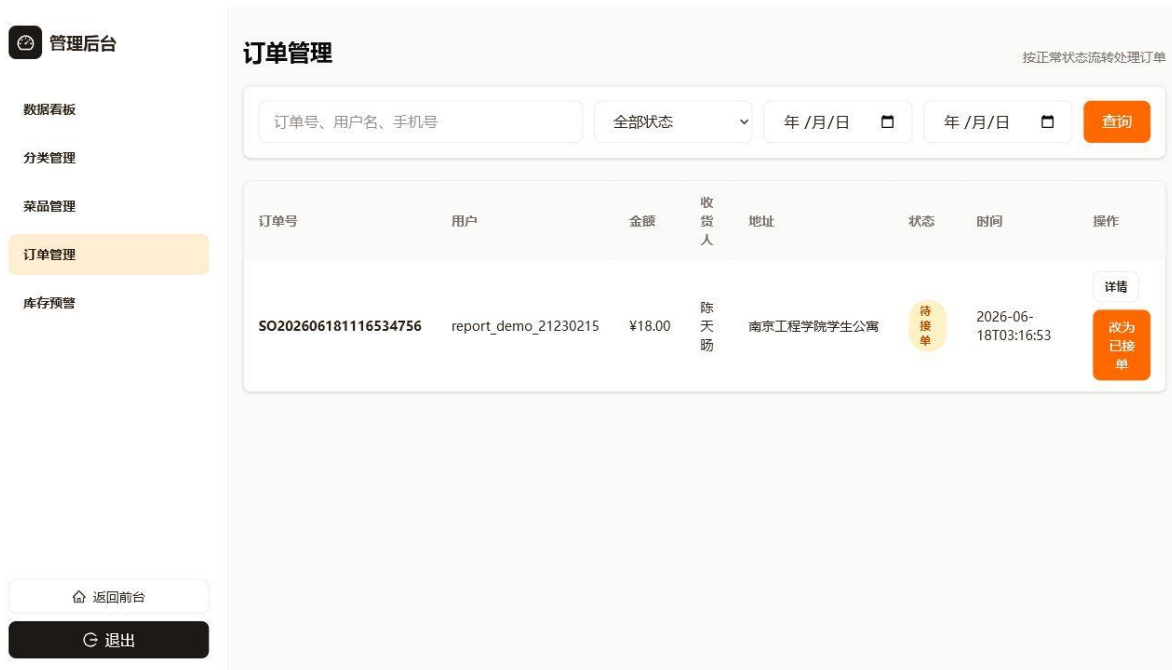


图 18 订单管理：查看所有客户订单并推动状态流转



图 19 库存预警：提示库存不足或需要补货的菜品

后台模块用于管理员维护经营数据。分类管理中，如果分类下已有菜品，后端会阻止删除；菜品管理中，如果菜品已经出现在订单明细中，则删除操作会转为下架。这两个规则都是为了保护数据库关联数据，避免历史订单和菜品分类出现断裂。

```
@Transactional
public boolean delete(Long id) {
    if (dishMapper.findById(id) == null) {
        throw new BusinessException(404, "菜品不存在");
    }
}
```

```
}
if (dishMapper.countOrderItems(id) > 0) {
    dishMapper.updateStatus(id, 0);
    return false;
}
dishMapper.delete(id);
return true;
}
```

管理员订单管理支持按订单号、用户名、手机号、状态和日期查询订单。订单状态必须按流程流转，不能跳过中间状态，这样可以模拟真实外卖业务中接单、制作、配送、完成的过程。

(5) 调试过程中的问题及解决方法

问题	原因分析	解决方法
前端跨域请求失败	前端运行在 5173，后端运行在 8090，浏览器会触发跨域限制	后端配置 CORS，允许本地前端访问接口
登录后后台接口仍提示无权限	请求头未正确携带 JWT 或用户角色不是 ADMIN	在 Axios 拦截器中统一加入 Authorization，并在后端 requireAdmin 校验角色
订单提交后库存不同步	只写入订单但没有扣减 dish.stock	下单事务中同时扣减库存、增加销量、清空购物车
已有订单菜品被删除后历史订单不完整	订单明细引用 dish_id，直接删除会影响数据关联	如果菜品已有订单，删除改为下架
截图中订单页面为空	演示数据库没有客户订单	通过演示客户提交订单后再截图，保证报告展示完整业务闭环

表 5 调试问题及解决方法

(6) 程序的不足与改进

当前系统已经完成课程要求的核心业务，但仍有改进空间。首先，系统还没有接入真实支付能力，订单金额只是用于业务展示；其次，评价功能、配送员端和商家多门店管理还可以继续扩展；第三，管理员权限目前只有 ADMIN 和 CUSTOMER 两类，后续可以拆分为菜品管理员、订单管理员和统计查看员；最后，项目目前主要面向桌面端展示，移动端适配还可以进一步优化。

如果继续完善，我会优先增加评价模块、支付状态、配送员端和更细粒度的权限控制，并将系统部署到服务器，配置生产环境数据库和文件上传路径，使项目从课程设计进一步接近真实可用系统。

六、课程设计收获体会

通过本次《WEB 开发技术》期末大作业，我完整经历了一个 Web 项目从需求分析、数据库设计、接口设计、前端页面实现、后端业务实现、联调测试到报告整理的过程。相比只写单个页面或单个接口，一个完整外卖点餐系统更强调模块之间的数据流转。例如客户从菜品列表加入购物车，购物车生成订单，订单扣减库存，管理员修改订单状态，客户再查看订单进度，这些步骤必须通过数据库和接口串联起来，任何一个环节设计不清楚都会影响整体体验。

在后端实现中，我更深刻地理解了分层设计的意义。Controller 只负责接收请求和返回响应，Service 负责业务规则，Mapper 负责数据库访问。这样写虽然比把所有逻辑堆在一个类里更繁琐，但当业务变复杂时优势非常明显。例如“已有订单的菜品只能下架不能删除”这个需求，只需要在 DishService 的删除逻辑中增加订单明细数量判断，就可以同时保护接口和前端操作。

在数据库设计方面，我认识到订单类系统不能只保存当前菜品信息，还要保存订单当时的菜品快照。如果订单明细只依赖 dish 表，一旦管理员修改菜品名称或价格，历史订单就会失真。因此我在 order_item 表中保存 dish_name、dish_price、quantity、subtotal 等字段，保证订单历史记录稳定。这种设计让我理解了数据库不仅要满足当前页面展示，还要考虑未来数据追溯和业务安全。

在前端实现中，我体会到用户体验不仅是页面好看，还包括流程是否清楚、反馈是否及时。例如未登录用户加入购物车时跳转登录，订单详情中使用时间轴展示进度，管理员后台用状态标签区分上架、下架、待接单和已完成等状态。这些细节能让系统更容易理解，也更适合验收展示。

本次项目也让我发现自己在项目规划和调试上的不足。最开始如果只关注页面效果，很容易忽略权限校验、异常提示和数据一致性；后面补充管理员注册码、菜品删除保护和订单状态日志后，系统的完整性明显提升。以后再做 Web 项目时，我会更早地从业务规则、数据模型和异常场景出发，再设计页面和接口。总体来说，本次课程设计让我把 HTML、CSS、JavaScript、Ajax、Java 后端和数据库知识串成了一个完整实践，对 Web 应用开发流程有了更系统的认识。

七、主要参考资料

- [1] 张娜. Java Web 开发技术教程（第三版）[M]. 北京：清华大学出版社，2023.
- [2] 黑马程序员. JavaEE 企业级应用开发教程（Spring+SpringMVC+MyBatis）[M]. 北京：人民邮电出版社，2017.
- [3] 孙卫琴. Tomcat 与 Java Web 开发技术详解（第三版）[M]. 北京：电子工业出版社，2019.
- [4] Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/>.
- [5] MyBatis 参考文档. <https://mybatis.org/mybatis-3/>.
- [6] React Documentation. <https://react.dev/>.

期末大作业成绩评价

动手能力（40%）	
创新能力（10%）	
大作业总结报告（20%）	
总评成绩	
大作业综合成绩评定结论	

以上评价栏保留给任课教师填写。